VERİ BİLİMCİ MÜFREDATI 2026

06

VERİ TEMİZLEME

Raw veriden güvenilir analitik veriye

Bu modülün hedefi

Eksik, tekrar eden, hatalı, tutarsız veya iş kurallarını ihlal eden veriyi sistematik biçimde tespit etmek; düzeltmek; doğrulamak ve temizleme kararlarını tekrar üretilebilir hale getirmek.

Profiling • Missing Values • Duplicates • Types • Outliers • Validation • Pandas • SQL

1. Veri temizleme nedir?

Veri temizleme, bir veri setini sadece “güzel görünür” hale getirme işi değildir. Amaç, analizin ve modelin kullandığı değerlerin iş anlamı, veri tipi, zaman bağlamı ve ilişki bütünlüğü açısından güvenilir olmasını sağlamaktır. Kötü veri; doğru yazılmış SQL, doğru istatistik ve güçlü bir modelin bile yanlış sonuç üretmesine neden olur.

> **Temel ilke**
> Önce veriyi anla ve ölç. Sonra temizle. Temizledikten sonra tekrar ölç. “Düzeltme” yaptığın her adımın gerekçesi ve etkisi görünür olmalı.

Veri temizleme ile veri dönüşümü aynı şey değildir

İşlem	Amaç	Örnek
Temizleme	Hatalı veya güvenilmez veriyi düzeltmek	-5 yaş değerini invalid kabul etmek
Dönüşüm	Analize uygun gösterim üretmek	Geliri log dönüşümüne sokmak
Feature engineering	Yeni bilgi taşıyan değişken üretmek	order_date -> customer_tenure_days
Imputation	Eksikliği kontrollü biçimde ele almak	median ile yaş doldurmak
Standardizasyon	Aynı anlamı tek formata getirmek	istanbul / İstanbul / IST -> İstanbul

Neden kritik?

- Yanlış KPI üretimini önler. Örneğin duplicate siparişler geliri iki kez saydırabilir.
- Model performansını ve genellenebilirliği etkiler. Hatalı kategoriler gereksiz feature seviyeleri oluşturur.
- A/B test, cohort, churn ve retention gibi zaman duyarlı analizlerde veri tanımını korur.
- Production pipeline içinde veri kalitesi hatalarını erkenden yakalamayı sağlar.

2. Veri kalitesi boyutları

Bir veri setinin “temiz” olup olmadığını tek bir sayı anlatmaz. Veri kalitesini farklı boyutlarda değerlendirmek gerekir.

Boyut	Soru	Örnek kontrol
Completeness	Gerekli değerler mevcut mu?	customer_id NULL oranı
Validity	Değer tanımlı kurala uyuyor mu?	discount_rate 0-1 aralığında mı?
Accuracy	Değer gerçeğe yakın/doğru mu?	ürün fiyatı kaynak sistemle tutarlı mı?
Consistency	Aynı bilgi farklı yerlerde çelişiyor mu?	order_total = item toplamı + kargo?
Uniqueness	Tekil olması gereken kayıt tekil mi?	order_id duplicate mı?
Timeliness	Veri yeterince güncel mi?	event ingestion gecikmesi
Integrity	Tablolar arası ilişki korunuyor mu?	orders.customer_id -> customers

İyi bir veri kalite raporu

```
dataset: orders
row_count: 2,485,120
duplicate_order_id: 0
null_customer_id_rate: 0.12%
invalid_quantity_rate: 0.03%
negative_revenue_rows: 148
orphan_customer_keys: 27
latest_event_time: 2026-10-09 23:58:41
```

Bu rapor yalnızca “kaç hata var?” sorusunu değil, hatanın hangi kurala göre tanımlandığını ve operasyonel olarak kabul edilebilir olup olmadığını da tartışmaya açar.

3. Temizliğe başlamadan önce: veri profiling

En yaygın hata, veri setini görür görmez null doldurmaya veya outlier silmeye başlamaktır. İlk adım profiling olmalıdır: veri setinin yapısını, dağılımını ve kalite sinyallerini ölç.

İlk 10 dakika kontrolü

1. Satır ve sütun sayısını kontrol et.
2. Her sütunun dtype bilgisini ve beklenen semantiğini yaz.
3. Unique/null oranlarını incele.
4. Sayısal sütunlarda min, max, median ve quantile değerlerini gör.
5. Kategorik sütunlarda top değerler ve rare kategorileri kontrol et.
6. ID alanlarında uniqueness ve referential integrity kontrol et.
7. Tarih alanlarında min/max zaman ve beklenmeyen gelecek tarihleri kontrol et.

Pandas profiling başlangıcı

```
import pandas as pd

df.shape
df.info()
df.head()
df.describe(include='all').T

profile = pd.DataFrame({
    'dtype': df.dtypes.astype(str),
    'null_count': df.isna().sum(),
    'null_rate': df.isna().mean(),
    'n_unique': df.nunique(dropna=False)
})
print(profile.sort_values('null_rate', ascending=False))
```

> **Soru**
> Bir sütunda %40 null görmek tek başına o sütunu silmek için gerekçe değildir. Önce null değerlerin mekanizmasını ve iş anlamını anlamalısın.

4. Eksik değerler (Missing Values)

Eksik değer “bilgi yok” anlamına gelebilir; ancak “uygulanamaz”, “henüz oluşmadı”, “ölçülemedi”, “tracking bozuk” veya “kullanıcı paylaşmadı” gibi farklı nedenleri olabilir. Bu nedenler aynı şekilde ele alınmamalıdır.

MCAR, MAR, MNAR sezgisi

Tür	Kısa tanım	Örnek
MCAR	Eksiklik diğer gözlemlerden bağımsızdır	Rastgele sensör paketi kaybı
MAR	Eksiklik gözlenen başka değişkenlerle ilişkili olabilir	Yaş bilgisi belirli kanallarda daha az dolduruluyor
MNAR	Eksiklik kendi gerçek değeriyle ilişkili olabilir	Yüksek gelirliler gelir bilgisini paylaşmıyor olabilir

Eksikliği ele alma seçenekleri

Yaklaşım	Ne zaman düşünülebilir?	Risk
Satır silme	Eksiklik çok az ve rastgele ise	Sample bias
Sütun silme	İş değeri zayıf, eksiklik çok yüksek ise	Bilgi kaybı
Sabit değer	“Unknown/Not applicable” anlamlı ise	Yapay kategori
Median/mean	Sayısal ve basit baseline ise	Varyansı bastırabilir
Group-based	Segment farkları anlamlıysa	Yanlış grup kuralı
Model tabanlı	Karmaşık yapı varsa	Leakage/overfit
Missing indicator	Eksikliğin kendisi sinyal taşıyorsa	Sinyali yanlış yorumlama

```
# Eksikliği ölç
df['age'].isna().mean()

# Median + indicator
df['age_missing'] = df['age'].isna().astype('int8')
df['age'] = df['age'].fillna(df['age'].median())

# Kategorik
df['channel'] = df['channel'].fillna('UNKNOWN')
```

5. Duplicate kayıtlar ve entity tekilliği

Duplicate problemi sadece satırların birebir aynı olması değildir. Aynı sipariş, müşteri veya event birden fazla kayda dönüşmüş olabilir. “Duplicate” tanımı verinin grain seviyesine göre yapılmalıdır.

Üç farklı duplicate tipi

Tip	Örnek	Doğru soru
Exact duplicate	Tüm kolonlar aynı iki satır	Ingestion iki kez mi yazdı?
Key duplicate	Aynı order_id birden fazla satır	Sipariş grain mi, item grain mi?
Near duplicate	İsim/adres küçük farklarla tekrar	Aynı gerçek dünya entity’si mi?

```
# Exact duplicate
df.duplicated().sum()

# İş anahtarına göre duplicate
dup = df[df.duplicated('order_id', keep=False)]

# Tek kayıt seçilecekse kural açık olmalı
df = (df.sort_values('updated_at')
        .drop_duplicates('order_id', keep='last'))
```

> **Kritik hata**
>
> `drop_duplicates()` çağrısını gerekçesiz kullanma. Hangi anahtara göre, hangi kaydı neden tuttuğunu açıklayabilmelisin. Aksi halde gerçek işlemleri silebilirsin.

6. Veri tipi ve schema problemleri

Bir sütunun teknik dtype bilgisi, iş tipini garanti etmez. `object` görünen bir tarih, string olarak gelen para, integer olması gereken müşteri ID'si veya kategorik bir alan yanlış dtype ile analizi sessizce bozabilir.

Sık görülen hatalar

- "1,249.90" gibi para değerlerinin string gelmesi.
- Tarihlerin DD/MM/YYYY ve MM/DD/YYYY karışması.
- ID alanlarının float'a dönüşüp 12345.0 görünmesi.
- Boolean alanların Y/N, true/false, 1/0 biçimlerinde karışık gelmesi.
- Kategori kodlarının sayısal görünmesine rağmen matematiksel anlam taşımaması.

```
# Güvenli dönüşüm örnekleri
df['order_date'] = pd.to_datetime(
    df['order_date'], errors='coerce', utc=True
)

df['price'] = pd.to_numeric(
    df['price'].str.replace(',', '', regex=False),
    errors='coerce'
)

df['customer_id'] = df['customer_id'].astype('string')
```

Schema validation düşüncesi

Alan	Beklenen kural
order_id	NOT NULL, unique at order grain
customer_id	NOT NULL, customers tablosunda bulunmalı
quantity	integer ve > 0
unit_price	numeric ve >= 0
order_date	timestamp, bugünden ileri olmamalı
status	izin verilen enum kümesinde olmalı

7. Invalid değerler ve iş kuralları

Temizleme kuralları çoğu zaman istatistikten değil iş mantığından gelir. Bir değer dağılım içinde “normal” görünebilir ama domain açısından imkansız olabilir.

Kural türleri

Kural	Örnek
Range	0 <= discount_rate <= 1
Set / enum	status IN (paid, shipped, cancelled, returned)
Cross-field	shipped_at >= paid_at
Conditional	status=returned ise return_date boş olmamalı
Aggregate	order_total yaklaşık item toplamına eşit olmalı
Temporal	event_time ingestion_time’dan makul ölçüde önce olmalı

```
invalid = df[
    (df['quantity'] <= 0) |
    (df['unit_price'] < 0) |
    (~df['status'].isin(['paid','shipped','cancelled','returned']))
]

# Cross-field rule
bad_time = df[
    df['shipped_at'].notna() &
    (df['shipped_at'] < df['paid_at'])
]
```

> **Silmek son seçenek olabilir**
>
> Invalid kayıtları otomatik silmeden önce karantina tablosuna ayırmak, kaynağı araştırmak ve hata oranını raporlamak üretim sistemlerinde daha güvenli bir yaklaşımdır.

8. Kategorik veri standardizasyonu

Aynı kavramın farklı yazımlarla tutulması grup analizlerini parçalar. Özellikle ülke, şehir, kanal, ürün kategorisi, cihaz, kampanya ve durum alanlarında standardizasyon kritiktir.

```
# Önce normalize et
s = (df['city']
     .astype('string')
     .str.strip()
     .str.casefold())

city_map = {
    'istanbul': 'İstanbul',
    'istanbul ': 'İstanbul',
    'ist': 'İstanbul',
    'ankara': 'Ankara'
}

df['city_clean'] = s.map(city_map).fillna(df['city'])
```

Standardizasyon adımları

1. Whitespace, case ve Unicode farklarını normalize et.
2. Canonical değer kümesini belirle.
3. Mapping tablosunu kod içine gömmek yerine mümkünse ayrı ve versiyonlu tut.
4. Map edilemeyen değerleri UNKNOWN yapmak yerine önce raporla.
5. Rare kategorileri sadece iş gereği varsa birleştir.

> **Neden mapping tablosu?**
>
> Kategori düzeltmeleri zamanla değişebilir. Kaynağı, eski değeri, yeni değeri, geçerlilik tarihini ve kural sahibini tutan mapping tabloları veri yönetişimini kolaylaştırır.

9. Aykırı değerler (Outliers)

Outlier her zaman hata değildir. Bir müşterinin 100 kat fazla harcaması VIP müşteri olabilir; sensörün -999 değeri ise hata kodu olabilir. Önce outlier'ın "nadir ama gerçek" mi yoksa "veri problemi" mi olduğunu ayır.

Tespit yöntemleri

Yöntem	Mantık	Dikkat
Domain rule	İş sınırı	En güçlü yaklaşım olabilir
IQR	Q1 - 1.5*IQR / Q3 + 1.5*IQR	Skewed dağılımda fazla işaretleyebilir
Z-score	Ortalamadan standart sapma uzaklığı	Normal dağılım varsayımına duyarlı
Robust z-score	Median + MAD	Outlier'a daha dayanıklı
Quantile cap	Alt/üst yüzdelik sınırı	Gerçek uç değerleri bastırabilir

```
q1 = df['revenue'].quantile(0.25)
q3 = df['revenue'].quantile(0.75)
iqr = q3 - q1
low = q1 - 1.5 * iqr
high = q3 + 1.5 * iqr

outliers = df[(df['revenue'] < low) | (df['revenue'] > high)]
```

Ne yapabilirsin?

- Hata ise kaynağını düzelt veya karantinaya al.
- Gerçek ve önemli ise koru.
- Model için gerekiyorsa log transform, robust scaling veya winsorization düşün.
- Silme/cap işleminin KPI ve model üzerindeki etkisini önce/sonra karşılaştır.

10. Metin, tarih ve zaman temizliği

Metin alanları

Metin temizliği kullanım amacına göre yapılmalıdır. İsim/adres standardizasyonu ile NLP metin preprocessing aynı problem değildir.

```
df['product_name_clean'] = (df['product_name']
    .astype('string')
    .str.strip()
    .str.replace(r'\s+', ' ', regex=True))
```

Tarih alanları

- Formatı açıkça belirle; ambiguous tarihleri varsayımla parse etme.
- Timezone bilgisini koru. Farklı bölgelerde UTC depolamak genellikle güvenlidir.
- Gelecek tarihleri, çok eski tarihleri ve sıra ihlallerini kontrol et.
- DST geçişleri ve timezone dönüşümlerinin günlük KPI'ları etkileyebileceğini unutma.

```
df['event_time'] = pd.to_datetime(
    df['event_time'], errors='coerce', utc=True
)

# Örnek temporal validity
now = pd.Timestamp.now(tz='UTC')
future_rows = df[df['event_time'] > now]
```

Tarih temizliğinde iş tanımı

"Günlük satış" hangi timezone'a göre hesaplanıyor? Sipariş tarihi created_at mı, paid_at mı? İptal edilen sipariş hangi güne yazılıyor? Temizlik ve KPI tanımı birlikte düşünülmelidir.

11. Join ve referential integrity problemleri

Birçok veri hatası tek bir tabloda görünmez; tablolar birleştirildiğinde ortaya çıkar. Yanlış join anahtarı, one-to-many çoğalma veya orphan key KPI'ları ciddi biçimde bozabilir.

Join öncesi kontrol

```
# customer_id customers'ta unique mi?
customers['customer_id'].is_unique

# Orders içindeki customer_id'lerin kaçı eşleşmiyor?
orphan = orders.loc[
    ~orders['customer_id'].isin(customers['customer_id'])
]

# Pandas ile cardinality validation
merged = orders.merge(
    customers,
    on='customer_id',
    how='left',
    validate='many_to_one'
)
```

> **Sessiz hata örneği**
>
> customers tablosunda aynı customer_id iki kez varsa her order iki satıra çoğalabilir. Toplam gelir artar ama kod hata vermez. Bu yüzden join cardinality kontrolü veri temizliğinin parçasıdır.

SQL kontrolü

```
-- Orphan customer key
SELECT COUNT(*) AS orphan_rows
FROM orders o
LEFT JOIN customers c
  ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;

-- Boyut tablosunda duplicate key
SELECT customer_id, COUNT(*) AS n
FROM customers
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

12. Temizleme pipeline tasarımı

Notebook içinde manuel düzeltmeler yapmak keşif aşamasında hızlıdır; fakat üretilebilir bir proje için temizleme mantığını fonksiyonlara, testlere ve veri kalite kontrollerine dönüştürmek gerekir.

```
RAW DATA
   |
   v
Schema / type checks
   |
   v
Standardization
   |
   v
Missing / duplicate / invalid rules
   |
   v
Referential integrity
   |
   v
Business rule validation
   |
   v
CLEAN DATA + QUALITY REPORT + QUARANTINE
```

Fonksiyonel Pandas örneği

```
def clean_orders(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    out.columns = out.columns.str.strip().str.lower()
    out['order_date'] = pd.to_datetime(out['order_date'], errors='coerce', utc=True)
    out['quantity'] = pd.to_numeric(out['quantity'], errors='coerce')
    out['status'] = out['status'].astype('string').str.strip().str.casefold()
    out = out.drop_duplicates('order_id', keep='last')
    return out

def validate_orders(df):
    assert df['order_id'].notna().all()
    assert df['order_id'].is_unique
    assert (df['quantity'].dropna() > 0).all()
```

Gerçek sistemlerde `assert` yerine Great Expectations, pandera, dbt tests veya özel kalite framework'leri kullanılabilir; önemli olan kuralın açık ve otomatik kontrol edilebilir olmasıdır.

13. Pandas ile pratik temizleme kalıpları

Problem	Pandas kalıbı
Kolon isimleri	df.columns = df.columns.str.strip().str.lower()
Whitespace	s.str.strip()
Multiple spaces	s.str.replace(r'\s+', ' ', regex=True)
String -> numeric	pd.to_numeric(s, errors='coerce')
String -> datetime	pd.to_datetime(s, errors='coerce', utc=True)
Null oranı	df.isna().mean()
Duplicate key	df.duplicated('id', keep=False)
Allowed values	s.isin(allowed)
Out of range	~s.between(low, high)
Rare categories	s.value_counts(normalize=True)

Method chaining örneği

```
clean = (raw
    .rename(columns=str.lower)
    .assign(
        order_date=lambda x: pd.to_datetime(x['order_date'], errors='coerce', utc=True),
        quantity=lambda x: pd.to_numeric(x['quantity'], errors='coerce')
    )
    .drop_duplicates('order_id', keep='last')
    .query('quantity > 0')
)
```

> **Okunabilirlik**
> Bir zincir çok karmaşık hale gelirse ara değişkenler kullan. “Tek satırda yazmak” temiz kod demek değildir.

14. SQL ile veri kalite ve temizleme kontrolleri

Null ve uniqueness

```
SELECT
  COUNT(*) AS rows,
  SUM(CASE WHEN order_id IS NULL THEN 1 ELSE 0 END) AS null_order_id,
  COUNT(DISTINCT order_id) AS distinct_orders
FROM orders;
```

Geçersiz değerler

```
SELECT *
FROM orders
WHERE quantity <= 0
   OR unit_price < 0
   OR status NOT IN ('paid','shipped','cancelled','returned');
```

Duplicate iş anahtarı

```
SELECT order_id, COUNT(*) AS n
FROM orders
GROUP BY order_id
HAVING COUNT(*) > 1
ORDER BY n DESC;
```

Cross-field tutarlılık

```
SELECT *
FROM orders
WHERE shipped_at IS NOT NULL
  AND paid_at IS NOT NULL
  AND shipped_at < paid_at;
```

SQL temizleme adımlarında `UPDATE`/`DELETE` çalıştırmadan önce aynı koşulu `SELECT` ile gözden geçirmek; etkilenen satır sayısını kaydetmek ve mümkünse raw veriyi değiştirmek yerine temiz bir downstream tablo üretmek daha güvenlidir.

15. Gerçek vaka: E-Commerce sipariş verisi

Bir e-ticaret ekibi aylık revenue dashboard'unda ani %9 artış görüyor. Pazarlama veya fiyat değişikliği yok. Veri ekibi önce "iş büyüdü" demek yerine veri kalitesini sorguluyor.

Ham veri sorunları

Sorun	Bulgu	Olası etki
Duplicate order_id	1,842 sipariş iki kez ingest edilmiş	Revenue şişer
status varyasyonu	PAID / paid / Paid / complete	Filtre eksik çalışır
quantity <= 0	İade/adjustment satırları karışmış	Adet ve gelir bozulur
unit_price string	Bazı satırlarda para sembolü var	Numeric hesap bozulur
Orphan customer_id	CRM sync gecikmesi	Segment analizi eksik kalır

Temizleme stratejisi

1. Raw tabloyu immutable tut.
2. Order grain ve item grain ayrımını netleştir.
3. Event timestamp'e göre duplicate ingestion kayıtlarını ayır.
4. Status değerlerini canonical mapping ile standardize et.
5. İade satırlarını "hata" diye silmek yerine business event olarak doğru modele taşı.
6. Fiyatı parse edilemeyen kayıtları quarantine tablosuna al.
7. Temizlik sonrası revenue ve order count KPI'larını raw veriyle reconcile et.

> **Sonuç**
> Temizlik sonrası gerçek revenue artışı %1.4 çıkarsa veri temizleme yalnızca teknik bir süreç değil, yanlış iş kararını önleyen analitik bir kontroldür.

16. Önce/sonra validasyon ve audit trail

Temizlik bittiğinde “kod çalıştı” demek yeterli değildir. Temizlemenin ne değiştirdiğini ölçmelisin.

Metrik	Raw	Clean	Yorum
Rows	1,250,000	1,247,982	2,018 duplicate/quarantine
Unique order_id	1,248,100	1,247,982	Tekillik sağlandı
Null order_date	312	0	Parse edilemeyenler karantinaya alındı
Invalid quantity	486	0	Rule uygulanmış
Revenue	৳84.2M	৳78.6M	Duplicate/invalid etkisi kaldırıldı

Audit trail’de ne tutulmalı?

- Pipeline/run ID ve çalışma zamanı.
- Kaynak dataset/version.
- Uygulanan kural veya kod sürümü.
- Her kuralın etkilediği satır sayısı.
- Karantinaya alınan kayıtların nedeni.
- Önce/sonra kalite metrikleri.

> **Reproducibility**
> Manuel Excel düzeltmeleri veya notebook hücresinde tekrar üretilemeyen değişiklikler, özellikle ekip ortamında veri güvenilirliğini zayıflatır.

17. Sık yapılan hatalar

Hata	Neden riskli?	Daha iyi yaklaşım
Null olan tüm satırları silmek	Bias ve veri kaybı	Eksiklik mekanizmasını analiz et
Tüm outlier'ları silmek	Gerçek yüksek değerli gözlemler kaybolabilir	Domain + robust analiz
drop_duplicates() kör kullanmak	Gerçek event/item satırları silinebilir	Grain + business key belirle
Raw veriyi overwrite etmek	Geri dönüş/audit zorlaşır	Raw immutable, clean ayrı
Join sonrası row count bakmamak	Revenue sessizce çoğalabilir	Cardinality validate et
errors="coerce" sonrası kontrol etmemek	Parse hataları NaN'a dönüşüp gizlenir	Coerce edilen oranı raporla
Temizleme kararını dokümante etmemek	Tekrar üretilemez	Rule + reason + impact logla

Temizleme mi, gizleme mi?

Bir veri kalite problemi upstream sistemde çözülmesi gerekirken downstream'de sürekli "patch" ediliyorsa, temizleme teknik borcu gizliyor olabilir. İyi veri bilimci yalnızca semptomu düzeltmez; hatanın kaynağını da görünür kılar.

18. Mülakat soruları ve kısa cevap çerçeveleri

Soru	İyi cevapta beklenen
Missing value gördüğünde ilk ne yaparsın?	Oranını, segmentlere göre desenini ve iş anlamını incelerim; sonra silme/imputation kararını gerekçelendiririm.
Mean mi median mı?	Dağılım ve outlier yapısına bağlı. Skewed sayısal dağılımda median daha robust olabilir; ama ikisi de otomatik doğru değildir.
Outlier nasıl ele alınır?	Önce hata mı gerçek gözlem mi ayrılır. Domain kuralı, IQR/MAD ve segment bazlı analiz birlikte kullanılır.
Duplicate nasıl tanımlanır?	Dataset grain ve business key üzerinden. Exact duplicate ile key duplicate farklıdır.
Join sonrası veri neden büyür?	One-to-many veya many-to-many eşleşme satırları çoğaltır. Anahtar uniqueness/cardinality kontrol edilmelidir.
Raw veriyi temizleyip üzerine yazar mısın?	Genellikle hayır. Raw immutable tutulur; temizlenmiş katman ayrı ve versioned üretilir.
errors="coerce" güvenli mi?	Kontrollü ise evet; parse edilemeyen değerlerin kaç tane olduğunu raporlamadan kullanmak tehlikelidir.
Veri temizleme leakage yaratabilir mi?	Evet. İmputation veya threshold değerleri tüm dataset üzerinden hesaplanıp train/test ayrımından önce uygulanırsa leakage olabilir.

19. Uygulama alıştırmaları

1. Bir `orders.csv` dosyasında dtype, null, unique ve min/max raporu üreten fonksiyon yaz.
2. `customer_id` için duplicate ve orphan key analizi yap.
3. `status` alanındaki tüm farklı yazımları canonical değerlere dönüştür.
4. `unit_price` içindeki para sembollerini temizleyip numeric'e dönüştür; parse edilemeyenleri raporla.
5. `order_date` ve `shipped_at` arasında temporal integrity testi yaz.
6. IQR ile revenue outlier'larını işaretle; silmeden önce segment bazında incele.
7. Temizlik öncesi/sonrası revenue, order count ve AOV değişimini raporla.
8. Temizleme kurallarını fonksiyonlaştır ve en az 5 test yaz.

Mini proje - Dirty E-Commerce Data Cleaner

Amaç: Ham e-ticaret sipariş verisini otomatik temizleyen, kalite raporu üreten ve karantina kayıtlarını ayrı saklayan tekrar üretilebilir bir pipeline oluştur.

```
project/
├── data/
│   ├── raw/orders.csv
│   ├── clean/orders_clean.parquet
│   └── quarantine/orders_invalid.csv
├── src/
│   ├── clean.py
│   ├── validate.py
│   └── quality_report.py
├── tests/
│   └── test_orders.py
├── notebooks/
│   └── 01_data_quality_analysis.ipynb
└── README.md
```

- README: problem, grain, data quality rules, before/after metrics.
- Kod: temizleme fonksiyonları + validation.
- Çıktı: clean dataset + quarantine + quality report.
- Portföy anlatısı: "hangi hatayı buldum, neden önemliydi, iş sonucunu nasıl etkiledi?"

20. Final kontrol listesi

- ☐ Dataset grain açıkça tanımlandı.
- ☐ Satır/sütun sayısı, dtype, null ve uniqueness profili çıkarıldı.
- ☐ Missing değerlerin nedeni ve treatment kararı belgelendi.
- ☐ Duplicate tanımı business key'e göre yapıldı.
- ☐ Numeric/date/bool/category tipleri doğrulandı.
- ☐ Range, enum ve cross-field business rule kontrolleri uygulandı.
- ☐ Kategori standardizasyonu canonical mapping ile yapıldı.
- ☐ Outlier'lar otomatik silinmeden önce incelendi.
- ☐ Join cardinality ve referential integrity doğrulandı.
- ☐ Raw veri immutable tutuldu.
- ☐ Temizleme adımları tekrar üretilebilir fonksiyon/pipeline haline getirildi.
- ☐ Önce/sonra kalite ve KPI etkisi ölçüldü.
- ☐ Karantina ve hata nedenleri saklandı.
- ☐ Testler ve audit trail mevcut.

> **Bu modülü bitirdiğinde**
>
> Bir veri setini sadece Pandas komutlarıyla "temizleyebilmek" değil; hangi kalite problemine hangi kuralı neden uyguladığını savunabilmek, etkisini ölçebilmek ve süreci tekrar üretilebilir hale getirebilmek hedeflenir.

Sonraki bağlantı

Veri temizleme, 7. modüldeki Exploratory Data Analysis için güvenilir taban oluşturur. EDA sırasında yeni kalite problemleri keşfedilebilir; bu nedenle veri temizleme ve EDA gerçek projelerde iteratif ilerler.